

20. File Storage & Sync (Google Drive / Dropbox)

Interviewer: Design Google Drive — or Dropbox, pick whichever mental model you like. Users upload files into a folder, and that folder needs to stay **in sync across every device they own**: laptop, phone, another laptop. Edit a file on one device, it shows up on the others. Go.

Candidate: Different flavor from a traffic-spike problem — there's no single hot key to defend. The hard part is **moving bytes efficiently between devices without re-sending whole files, and never losing an edit when two offline devices disagree**. Let me clarify first.

1. Clarifying requirements

Candidate: Quick questions: 1. Files can be large — videos, zips. If I change one paragraph in a 500MB doc, must we avoid re-uploading the whole 500MB? 2. How fast must sync feel — sub-second, or is a few seconds fine? 3. Can a device go **offline for days**, edit locally, then reconnect? If two devices edit the *same file* offline, what happens? 4. Do we need version history, or just "latest wins"?

Interviewer: Large files are the whole point — never re-send an untouched file. A few seconds of lag is fine. Offline edits for days are normal, and **nobody's edit should silently vanish** on conflict. Keep version history.

Candidate: Requirements:

Functional — upload/download, organize into folders; sync an edit to other devices within seconds; move only the *changed portion* of a large file, not the whole thing; survive multi-day offline edits and catch up efficiently on reconnect; never drop a conflicting edit; keep version history.

Non-functional — **durability** (trusted like a backup); **bandwidth efficiency** (the star of this problem); **eventual consistency across devices** but **zero data loss** on conflict; scale to hundreds of millions of users × multiple devices × thousands of files each.

2. Estimation

500M users, avg 10GB stored each → 5 PB raw (pre-dedup).
Avg file ~20MB, chunk size 4MB → ~5 chunks/file. 500M * 5,000 files = 2.5B files.

100M file-saves/day → ~1,150 writes/sec avg, ~10k/s at peak.

Naive whole-file re-upload: 1,150/s * 20MB avg file = ~23 GB/sec sustained upload.
Delta sync (only ~1 of 5 chunks changes on a typical edit): ~4.6 GB/sec → ~5x less, and 100x+ less for a small edit to a huge video file.

3 devices/user * 500M users = 1.5B long-lived notification connections (sharded across many notification servers).

Candidate: So this isn't a "protect one hot row" problem like the burger giveaway — it's a "**minimize bytes moved, never silently drop an edit**" problem. Everything below serves that.

3. A simple V1 (and why it breaks)

```
[Device] --PUT whole file--> [Upload Service] --> [Object Storage: file.docx]
                                     ↳ [Metadata DB: file_id → latest blob key]
Other devices POLL "has file_id changed?" every N seconds; download the
WHOLE new blob if so.
```

Why it dies: every tiny edit re-moves the *entire* file both ways — one changed character in a 2GB video re-sends 2GB. Polling adds needless lag or hammers the server. And "latest write wins" silently destroys a second device's offline edits — explicit data loss. Two fixes needed: stop treating a file as one atomic blob, and push instead of poll. The first is the core trick.

4. Chunking — never move a whole file

Interviewer: Walk me through avoiding a full re-upload.

Candidate: Split every file into **chunks** and address each by the **hash of its own bytes** (SHA-256). A file's identity becomes an ordered list of chunk hashes, not one blob.

```
A 12MB file, split into chunks:
+-----+-----+-----+
|  Chunk 1  |  Chunk 2  |  Chunk 3  |
| hash: h1a2b3 | hash: h4d5e6 | hash: h7f8g9 |
+-----+-----+-----+
Edit touches only the END → only Chunk 3 changes → new hash h9x9x9.
Chunks 1,2 byte-identical → unchanged hashes → upload ONLY Chunk 3.
New chunk_list = [h1a2b3, h4d5e6, h9x9x9]      ← DELTA SYNC
```

Upload negotiation:

1. Client hashes its chunks locally.
2. "Of [h1,h2,h3], which do you already have?"
3. Server: "have h1,h2 – need h3."
4. Client uploads only h3's bytes. 5. Client commits chunk_list=[h1,h2,h3].

Interviewer: Fixed 4MB chunks — what if I insert one byte at the *start* of the file?

Candidate: That's the classic weakness of **fixed-size chunking**: an insertion shifts every byte after it, so every boundary downstream moves and every hash changes — even though almost nothing actually changed content-wise.

Fix: content-defined chunking (CDC). A rolling hash (Rabin fingerprint) over a sliding window declares a boundary wherever the hash matches a pattern (e.g., low 13 bits zero, giving ~4MB average chunks) — boundaries move with the *content*, not the byte offset.

FIXED-SIZE (insert shifts everything):

before: [A A A A | B B B B | C C C C]

after: [X A A A A | A B B B B | B C C C C] ← every boundary moved, ALL new hashes

CONTENT-DEFINED (boundary re-aligns on content):

before: [A A A A | B B B B | C C C C]

after: [X A A A A | B B B B | C C C C] ← only chunk 1 changes; B, C re-align

Why CDC here: fixed-size is simpler and fine for pure appends, but general document/video edits involve insertions/deletions. CDC keeps delta sync effective whenever bytes shift, at the cost of a bit of client CPU for rolling hashes — worth it, since bandwidth is the whole point of this product.

5. Block-level dedup — free once chunks are content-addressed

Interviewer: Two users upload the same PDF. Stored twice?

Candidate: No — content-hashing gives dedup for free. If a new chunk's hash already exists anywhere in the store, we just bump a `ref_count` and point the new version at the existing chunk. Zero extra bytes.

CHUNK STORE (content-addressed, dedup'd)

```
hash h4d5e6 -> storage_key: blk/h4/h4d5e6.bin
               ref_count: 3  (3 versions point here)
```

▲ FileA v2 (User1) ▲ FileB v1 (User2) ▲ FileC v5 (User1, other file)

Same bytes stored once. Deleting a ref decrements `ref_count`; bytes are freed only when `ref_count` hits 0, and only by an async GC job.

Why chunk-level, not file-level dedup: file-level hashing only catches identical whole files (rare). Chunk-level catches the common case — a 200MB deck where one slide changed still shares ~190MB of

chunks with the previous version, and often with unrelated files that share boilerplate (templates, stock assets). That's where real storage savings come from.

6. Metadata service vs. block store

Interviewer: Where does all this live — one database?

Candidate: No — this split is the single most important architectural call here.

METADATA (small, structured, hot)	BLOCK STORE (huge, immutable, write-once)
File(id, name, owner_id, folder_path, current_version_id) FileVersion(version_id, file_id, created_at, chunk_list[]) Chunk(hash PK -> storage_key, size, ref_count) Device/SyncState(device_id, user_id, last_synced_version) -> sharded relational DB (Postgres/Aurora): small multi-row txns need ACID (rename + folder update together)	Object storage: key = chunk hash, value = raw bytes (~4MB avg) Replicated across disks/AZs by the store itself – durability "built in." -> S3/GCS-class blob store: cheap per GB, horizontally scalable to billions of objects; never put multi-MB blobs in DB rows

```
[ Client ] —metadata calls—> [ Metadata Service ] —> [ Metadata DB (sharded) ]
[ Client ] —chunk bytes—> [ Block Service ] —> [ Object Storage + CDN ]
```

Why split them: metadata is ~2.5B files × ~1KB ≈ a few TB — trivial for a sharded relational DB doing fast point lookups and small atomic commits. The block store holds the real 5PB of bytes and needs to be cheap-per-GB and horizontally scalable — not what a relational DB is built for. Mixing them (blobs-in-rows) bloats the DB and kills backup/scaling for no gain.

7. Sync mechanism — notify, then pull the delta

Interviewer: Device A commits a new version. How do B and C find out?

```
Device A commits v6 (chunk_list=[h1,h2,h9x9x9])
-> [Metadata Service] writes FileVersion row, publishes "file 42 -> v6"
-> [Pub/Sub, partitioned by user_id]
-> [Notification Service] fans out:
    Device B (open WebSocket): push "file 42 -> v6, sync now"
    Device C (offline): nothing to push, catches up on reconnect

Device B, on push:
1. Ask metadata service for v6's chunk_list.
2. Diff against locally-known chunk_list.
3. Download only the missing chunks (h9x9x9), often via CDN (chunks are immutable).
4. Reassemble the file from [h1(have), h2(have), h9x9x9(new)].
```

Why push, not poll: polling either feels laggy or, if aggressive, wastes battery/hammers the metadata service across hundreds of millions of idle devices. A WebSocket (long-poll fallback) lets the server push the instant a change lands.

Why fan out via pub/sub, not directly from the metadata service: a shared folder can have 500 collaborators. Pushing to 500 sockets synchronously inside the commit request would slow the write path and couple it to notification-delivery health. One published event + an independently scaled fan-out tier keeps the commit fast regardless of audience size.

8. Multi-region

Interviewer: A Tokyo user and a São Paulo user share a folder. How do you place things?

Candidate: Same metadata/bytes split applies. **Chunk bytes** live in object storage in each user's **home region** (lower latency, data-residency compliance), with a global CDN caching hot reads regardless of origin. **Metadata** also has a home region (usually the owner's) — writes go through that region's primary, with read replicas elsewhere for low-latency browsing. **Cross-region sharing** pays a one-time latency hit fetching bytes cross-region (then cached locally) rather than synchronously replicating every file into every region — most files are never shared cross-region, so that cost isn't worth paying globally.

9. Latency — where the ms matter, and where they don't

Candidate: Unusually relaxed here since seconds of sync lag is acceptable. Budget goes to **chunk upload/download** — signed URLs straight to object storage (skip our app servers) plus CDN for repeat reads, since a slow save blocks the "safe to close the laptop" moment — and the **"which chunks do you have" negotiation**, kept cheap via a Redis/Bloom-filter cache of hot chunk-existence checks so every upload doesn't hit the full Chunk table. Version history, usage stats, and GC are not urgent — batchy, unnoticed.

10. Consistency — conflict resolution on concurrent edits

Interviewer: Devices A and B are both offline, both edit the *same* file, both reconnect. What happens?

Candidate: The wrong answer is last-write-wins — that silently destroys someone's edit.

Approach: version check + conflict copy (what Dropbox actually does).

File at v5. A and B both synced v5, then went offline, each edited independently.

A reconnects first: `commit(base=v5, chunk_list=...)` -> current IS v5 -> no conflict
-> file becomes v6 (A's edit).

B reconnects second: `commit(base=v5, chunk_list=...)` -> current is ALREADY v6
-> CONFLICT.

Resolution: A's edit stays as v6 (the "real" file). B's edit is saved as a sibling: "report.docx (Device B's conflicted copy, Jul 19).docx". Both users notified. Nothing deleted; human merges manually if needed.

```
[v5] —A, reconnects first—> [v6 = A's edit]          (accepted)
    |—B, reconnects second, also based on v5—> [v6-conflict-B] (sibling file)
```

Why a version check instead of a lock: a lock held for hours/days of offline editing defeats the whole point of an offline-friendly sync product. The version check is a cheap single-row compare-and-set on the metadata primary, and only the *second* committer is "penalized" — with a conflict copy, never a rejection or data loss.

Strong vs. eventual: deciding "which version is current" is strong (single-row check-and-set on the primary — two commits can't both win v6). Every other device's *view* of the file is eventual, catching up within normal sync-notification lag.

11. Async — what leaves the critical path

Candidate: Critical path = upload missing chunk bytes + commit the FileVersion row. Everything else fans out behind it.

```
[Client commits v6] <- "saved" checkmark appears HERE (sync, must succeed)
-> [Metadata DB write: FileVersion + chunk_list]
    -> publish "file 42 -> v6"
        -> [Kafka: file-change-events]
            ├──> [Notification workers] -> WebSocket fan-out
            ├──> [Indexing workers]      -> search index
            ├──> [Dedup/GC workers]      -> ref_count-- on dropped chunks
            └──> [Thumbnail workers]      -> preview generation
```

Why safe async: "did my save succeed" is answered by the metadata commit alone (small, fast, no huge blob in the path). Notifying other devices, indexing, GC, thumbnails — nobody is staring at a spinner for those; decoupling them means a slow indexer can never make a save feel slow.

12. Caching

Chunk bytes behind a CDN — a chunk never changes once written (hash is its identity), so it's permanently cacheable with zero invalidation logic, and it absorbs viral-file read storms. **Chunk-existence checks in Redis/Bloom filter** — avoids hitting the Chunk table on every upload negotiation; also invalidation-free, since "this hash exists" never becomes false. **Hot file metadata in Redis** for popular shared folders — invalidated by the same change-event stream from section 11, so no second

invalidation mechanism is needed. The theme: almost everything cached here is **immutable**, which is why caching stays unusually simple.

13. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Chunk bytes (5PB, billions of blobs)	Object storage (S3-class)	Cheap per GB, horizontally scales to billions of immutable objects, built-in cross-AZ replication. <i>Not a database</i> — multi-MB blobs in DB rows bloat storage, slow backups, and don't scale past a few TB comfortably.
Chunking scheme	Content-defined chunking (rolling hash) over fixed-size	Boundaries re-align after insertions/deletions, keeping delta sync effective on real edits. <i>Not fixed-size</i> — a single inserted byte shifts every downstream boundary and invalidates every hash after it.
Dedup granularity	Chunk-level, content-addressed	Catches partial overlap (one changed slide in a 200MB deck, shared templates across users), not just exact-duplicate files. <i>Not file-level hashing</i> — misses the far more common "mostly-same-file" case.
Metadata (files, versions, chunk_lists)	Sharded relational DB (Postgres/Aurora)	Small, structured, needs ACID for multi-row commits (rename + folder update; "current version" check-and-set). <i>Not storing metadata in the object store</i> — no cheap transactional point-lookups there.
Sync notification	WebSocket/long-poll + pub/sub fan-out	Server pushes the instant a change lands; pub/sub decouples fan-out width (500-person shared folder) from the commit path's latency. <i>Not polling</i> — wastes battery/bandwidth across billions of idle devices or adds needless lag.
Event backbone	Kafka	Durable, partitioned, replayable — feeds notification, indexing, GC, and thumbnailing consumers independently. <i>Not direct synchronous calls from the commit path</i> — a slow indexer would otherwise slow every save.
Chunk-existence check cache	Redis / Bloom filter in front of the Chunk table	O(1) "have we seen this hash," no invalidation risk since existence is monotonic (never becomes false). <i>Not a DB lookup per hash per upload</i> — too much point-lookup load at scale.
Conflict handling	Version check + conflict copy , not locking	Keeps both edits, never silently drops one; a lock would block editing for the entire offline duration, which defeats an offline-first product.

Say-it-out-loud justification: "Object storage holds the actual bytes because they're huge, immutable, and need to scale to billions of blobs cheaply; a sharded relational DB holds the small, structured, transactional metadata because renaming a file or flipping 'current version' needs ACID, not blob-scale storage. Content-defined chunking plus content-addressed hashing gives me delta sync and dedup as two sides of the same coin. Sync is server-push through pub/sub, not client-poll, so it scales to hundreds of millions of idle devices and to wide-fan-out shared folders alike. And conflicts are never resolved by

picking a winner and discarding the loser — a version check plus a conflict copy guarantees no edit is ever silently lost, which is the one thing this product can never get wrong."

14. Failure modes & graceful degradation

Failure	What happens	Mitigation
Upload interrupted mid-file	Some chunks up, version uncommitted	Resumable — client re-asks "which chunks do you still need," sends only the gap
Notification service down	No instant push	Devices fall back to periodic polling; sync still completes, just laggier
Metadata DB primary down	Commits blocked	Failover to promoted replica; reads served by remaining replicas, slightly stale
Object storage region down	Chunk downloads fail	Store's own cross-AZ/region replication serves another copy; metadata unaffected
Device offline for days	Falls far behind	Catch-up sync: "what changed since my last version?" — pulls only the deltas
Simultaneous conflicting commits	Risk of silent data loss if mishandled	Version check + conflict copy (section 10) — both edits always survive

15. Follow-up curveballs

Interviewer: How small should a chunk be?

Candidate: Tradeoff: smaller chunks (~1MB) give finer delta sync but multiply metadata rows and negotiation round-trips; larger chunks (16MB+) mean less metadata overhead but force re-upload of a whole chunk for a tiny edit inside it. ~4MB is the common real-world compromise (roughly Dropbox's number).

Interviewer: How do you avoid re-hashing a whole multi-GB file on every tiny edit?

Candidate: Most clients know the dirty byte-range from the editor/OS and only recompute rolling-hash boundaries in a window around it, confirming re-convergence with untouched neighbors just past the edit. Worst case (no dirty-range hint) is a full local re-scan — client CPU cost, not network cost, so still far cheaper than re-uploading the file.

Interviewer: When are deleted chunks actually freed from storage?

Candidate: Never synchronously — deleting a version only decrements `ref_count`. A background GC job sweeps chunks at `ref_count = 0` and deletes the bytes. Nice side effect: a free "undo delete" window until GC actually runs.

Interviewer: How does sharing a folder with 500 people interact with this design?

Candidate: Sharing itself is a permissions row (`file_id` -> `user_id` -> `access_level`) checked before returning chunk locations or accepting a commit — orthogonal to chunking. What *does* stress the design is notification fan-out (500 devices instead of 3), which is exactly why fan-out lives in a separately-scaled pub/sub tier (section 7) rather than inside the commit path.

16. Deep-dive: "Why not just upload the whole file on every change?"

Interviewer: Chunking, content-addressing, delta sync — that's a lot of machinery. Why not keep it simple: every save, upload the whole file, store the whole file? Object storage is cheap. What's actually wrong with that?

Candidate: It's the right instinct to question complexity, and for tiny files it genuinely doesn't matter. But at this product's scale it breaks on three separate axes, and they compound.

Bandwidth waste — the killer for edits

A 1GB video file. User trims one frame / fixes one typo in an embedded caption track — net change: a few bytes.

Whole-file re-upload: must send the FULL 1,000,000,000 bytes, every save.

Chunked delta sync: ~4MB average chunk size -> only the touched chunk(s) resend.
1 changed chunk / ~250 chunks in the file -> ~4MB sent.

Ratio: 1,000MB vs ~4MB -> ~250x more bytes for whole-file, for a one-character edit.

At fleet scale (100M saves/day from section 2): if even 1% of saves touch a file this large, whole-file re-upload alone adds $\sim 1\text{M} \times 1\text{GB} = 1\text{ PB/day}$ of pure waste — bytes that carry zero new information.

This isn't a rounding error — it's the difference between "sync feels instant on Wi-Fi" and "sync visibly stalls, and mobile users on metered data revolt."

Storage cost — without dedup, every version is a full copy

Keep 10 versions of a 1GB file for version history:

Whole-file storage: 10 versions * 1GB = 10GB stored, even though 9 of the 10 versions are >99% byte-identical to their neighbor.

Chunk + dedup: ~250 shared chunks (unchanged) stored ONCE, ref_counted across all 10 versions; only the handful of chunks that actually changed per edit are net-new. Realistic total: often just 1-2x the base file size for ALL 10 versions combined, not 10x.

Multiply by 2.5B files and "keep version history" (a stated requirement) goes from "the feature that makes storage cost explode" to "nearly free," purely because of chunk-level content addressing.

Resume-on-failure — whole files force all-or-nothing uploads

Uploading a 2GB file over a flaky connection. It drops at 99% (1.98GB sent).

Whole-file: the object isn't valid until 100% arrives -> the 99% is thrown away -> restart from byte 0. On a bad connection this can mean NEVER finishing.

Chunked: each ~4MB chunk is a complete, independently-committed unit. On reconnect, client re-asks "which chunks do you still need?" and resends only the handful that never landed -> tiny amount of re-work, not 2GB.

This is the same mechanism failure-mode table in section 14 leans on for "upload interrupted mid-file" — it falls out of chunking for free, not as a bolted-on special case.

Concern	Whole-file upload	Chunked + dedup + delta sync
Bandwidth for a 1-char edit in a 1GB file	~1GB re-sent	~4MB (one chunk) — ~250x less
Storage for 10 versions of a mostly-unchanged file	10x the file size	~1-2x, via ref-counted shared chunks
Cross-user dedup (same file, two users)	None — two full copies	Automatic — identical chunk hashes collapse to one stored copy
Resume after a dropped connection	Restart from byte 0	Resend only the chunks that never landed
Implementation complexity	Trivial — one PUT	Higher — chunker, hash negotiation, chunk-existence cache, GC/ref-counting
Good fit for	Small files, infrequent writes, prototypes	Large files, frequent small edits, version history, multi-tenant dedup

Fixed-size vs content-defined chunking, briefly (detail in section 4): fixed-size chunks are simpler but an insertion shifts every downstream boundary, invalidating hashes it didn't need to; content-defined chunking re-aligns boundaries on the content itself, so only the chunk actually touched by an edit changes. Either beats whole-file re-upload by an order of magnitude — CDC just makes the delta *tighter*.

Rule of thumb: *Whole-file upload is the right call when files are small and rarely re-saved — the complexity of chunking isn't worth it. The moment "large files" + "frequent small edits" + "version history" all show up together, as they do here, chunked delta sync stops being an optimization and becomes the thing that makes the product viable at all.*

17. Deep-dive: two devices edit the same file offline — walked through in detail

Interviewer: Let's slow down on section 10. Walk me through the actual timeline, tick by tick — what does each device know, and exactly when does the system realize there's a conflict?

Candidate: Sure. The key fact to hold onto: **while offline, neither device knows the other is editing.** They're each happily editing their own local copy, believing it's still current.

The timeline

t=09:00 File "report.docx" is at v5. Laptop and Phone both synced v5 before going offline.

Laptop: [OFFLINE 09:00-10:32] --- edits paragraph 1 --- (still thinks base = v5)

Phone: [OFFLINE 09:05-10:35] --- edits paragraph 3 --- (still thinks base = v5)

Neither device can see the other. Both are making a perfectly reasonable, independent edit against what they each believe is "the current file."

t=10:32 Laptop reconnects. commit(base=v5, chunk_list=[...laptop's edit...])
-> server checks: current version IS v5 -> no conflict, fast-forward.
-> file becomes v6 (Laptop's edit).

t=10:35 Phone reconnects. commit(base=v5, chunk_list=[...phone's edit...])
-> server checks: current version is ALREADY v6 (not v5!) -> CONFLICT.
-> Phone's base is stale - it edited against a version that's no longer current.

Why this is a REAL conflict, not just "Phone is behind":

version vector view (per-device edit counters since the common ancestor v5):

Laptop's commit: { Laptop: 1, Phone: 0 }

Phone's commit: { Laptop: 0, Phone: 1 }

Neither vector dominates the other (each has a count the other lacks) -> the edits are CONCURRENT, not sequential. If Phone's vector had been {Laptop:1, Phone:0} (i.e. it had synced Laptop's v6 before editing), that WOULD be a simple fast-forward, no conflict. The divergence is what makes this a true conflict, not staleness.

Ranked resolution strategies

Strategy	How it works	Data-loss risk	Verdict here
1. Last-write-wins	Whoever commits last simply overwrites the file; loser's bytes are discarded.	High — Phone's entire edit vanishes silently, no notification.	Rejected — violates the stated "nobody's edit should silently vanish" requirement outright.
2. Keep both as a conflicted copy	Winner's commit becomes the canonical version; loser's edit is saved as a sibling file (report (Device B's conflicted copy) .docx). Both users notified.	None — both edits persist as files; human resolves manually if needed.	Recommended for this product. Simple, safe, works for <i>any</i> file type.
3. Operational Transform (OT) / CRDT merge	Represent edits as structured operations (insert/delete at a position, or a CRDT type) that can be algebraically merged/reordered so both edits land in one document automatically.	Low , but only within the operation model — a bad merge can still produce garbled content the user didn't intend.	Only works for structured, editable text/data models (what Google Docs does). We're syncing opaque byte blobs — PDFs, videos, zips — there's no generic "merge operation" for arbitrary binary content.

Strategy	How it works	Data-loss risk	Verdict here
4. Version vectors purely for detection	Track a causality vector per device; use it to <i>distinguish</i> true conflicts (diverged vectors) from simple staleness (one vector dominates, just fast-forward, no conflict needed).	N/A — this is detection, not resolution; still needs strategy 2 or 3 on top.	Already implicitly what the base=v5 check in section 10 does; making it an explicit per-device vector (rather than one linear version number) is the more correct generalization once you have >2 devices editing concurrently.

Recommended approach: version vectors (or the simpler single-version compare-and-set from section 10, which is the 2-device special case of the same idea) **to detect** the conflict, combined with **keep-both-as-conflicted-copy** to **resolve** it.

Why simple file sync picks conflicted-copy over auto-merge: OT/CRDT is the *better* answer, but only for a product that owns the document's structure end-to-end — a collaborative text editor where every keystroke is a known operation type. A file-sync product like Drive/Dropbox has to support **arbitrary binary files** — a PDF, a `.zip`, a video — where "merge these two versions" has no general, safe definition. Attempting a byte-level auto-merge on unknown file formats risks silently producing a **corrupted file that looks fine until someone opens it** — arguably worse than a conflict copy, because it fails silently instead of visibly. Handing the human two clearly-labeled, fully-intact files and letting them decide is the conservative, general-purpose choice; OT/CRDT is reserved for the narrower case (in-house collaborative editors on top of this same storage layer) where the document model is actually known.

What to say out loud: *"The two devices diverge because each edits against the same base version while offline and neither can see the other — that's exactly what a version vector makes precise: two diverged vectors mean a true concurrent conflict, not one device just being behind. For resolution, I'd keep both edits — one becomes the canonical version, the other a clearly-labeled conflicted copy — because we're syncing arbitrary binary files with no safe generic merge operation. OT or CRDTs are the right tool if this were a structured, real-time collaborative editor with a known document model, but for general file sync, never-silently-lose-data beats auto-merge every time."*

Summary — the mental model

File sync is **not** a hot-key contention problem — it's a **"move the fewest bytes possible, never silently lose an edit"** problem. The winning moves: **content-defined chunking** addressed by hash turns edits into **delta sync** and gives **dedup** for free; **metadata (small, structured, ACID) is strictly separated from chunk bytes (huge, immutable, cheap object storage)** — mixing them is the classic mistake; sync is **server-push through pub/sub fan-out**, not client-poll, so it scales to both idle devices and huge shared folders; and concurrent offline edits are resolved with a **version check that always keeps both edits** as a conflict copy, never a silent last-write-wins. Because almost everything here is immutable (chunk bytes, chunk existence), caching is trivial — the one genuinely hard question, "whose edit is v6," is answered by one atomic check on the metadata primary, not by locking a file for the days it might be offline.